

RicoSLAM: Framework for Localization and Mapping in Dynamic Environments

Ricardo B. Sousa^{1,2}, Giorgio Grisetti³, Héber Miguel Sobreira²,
Carlos André Silva⁴, and António Paulo Moreira^{1,2}

Abstract—Mobile robots deployed in industry must localize and map while people and moving elements move around them. In this paper, we address the problem of 2D LiDAR SLAM in such environments, where moving objects corrupt the map, wheel odometry is unreliable under slippage, and the pose graph grows with the duration of the run rather than with the explored space. Our approach builds the RicoSLAM framework around an Unsigned Distance Field (UDF)-based front-end, where UDF serves as the tracker’s registration target and accumulated scans in local maps are the source of the final occupancy grid map. A keyframe is committed once the tracker can no longer register against the current local map. Then, a relocalization policy reattaches it to existing nodes on revisits. Dynamic returns are discarded by ray tracing. We implemented our approach in C++ and ROS and evaluated it on the IILABS 3D dataset against GMapping, SLAM Toolbox, and Cartographer, and on two industrial datasets with dynamic observations. The experiments show that our odometry-free system is the only one below 0.05 m of Absolute Trajectory Error (ATE) on IILABS 3D, using 77 graph nodes where the most accurate baseline requires 543.

Index Terms—2D laser scanner, dynamics removal, ground mobile robots, unsigned distance fields, ray tracing, simultaneous localization and mapping (SLAM).

I. INTRODUCTION

Autonomous robots transport goods between production lines, warehouses, and loading docks. Their navigation stack depends on a consistent map of the environment, typically built with 2D lasers, which remains the reference sensor for safety-related functions of driverless industrial trucks [1]. However, these facilities are not static. Operators walk through the aisles, have moving machinery, and goods are relocated between shifts. So, each traversal mixes permanent structure with moving elements that will be gone on the next traversal.

In this paper, we address the 2D Simultaneous Localization and Mapping (SLAM) problem in such environments, where three difficulties arise. First, the static-world assumption. Returns from moving elements are integrated into the map,

¹Faculty of Engineering, University of Porto (FEUP). R. Dr. Roberto Frias, 4200-465 Porto, Portugal. {rbs, amoreira}@fe.up.pt

²INESC TEC – Institute for Systems and Computer Engineering, Technology and Science. R. Dr. Roberto Frias, 4200-465 Porto, Portugal. heber.m.sobreira@inesctec.pt

³Sapienza University of Rome. Piazzale Aldo Moro 5, 00185 Roma RM, Italy. grisetti@diag.uniroma1.it

⁴Flowbotic Mobile Systems, S.A. Lugar do Pombal, Zona Industrial de Salgueiro, 3530-259 Visu, Portugal. carlos.silva@flowbotic.eu

This work is co-financed by Component 5 – Capitalization and Business Innovation, integrated in the Resilience Dimension of the Recovery and Resilience Plan within the scope of the Recovery and Resilience Mechanism (MRR) of the European Union (EU), framed in the Next Generation EU, for the period 2021–2026, within project GreenAuto, with reference 54.

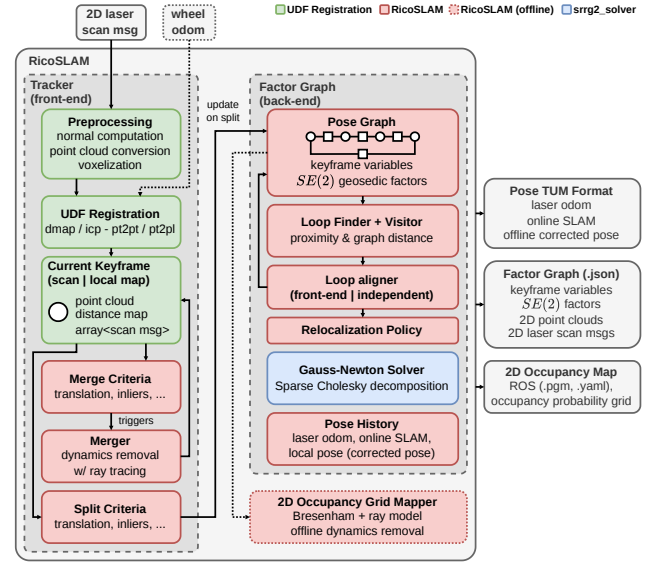


Fig. 1: Block diagram of the RicoSLAM framework.

degrading the scan registration reference. Second, the back-end growth. Pose graph formulations commit new variables as the robot moves. Thus, the graph scales with operation time rather than with the facility extent [2]. Third, the motion prior. SLAM systems commonly initialize scan registration with wheel odometry, which may be corrupted by wheel slippage that heavy payloads and abrupt maneuvers induce.

These difficulties are usually addressed by separate mechanisms. Dynamics are handled with dedicated map layers or temporal models [3], [4], by comparing aligned sessions [5], [6], or with trained classifiers [7], [8]. Graph growth is contained by pruning or marginalizing nodes that have already been created [9]–[11]. In contrast, this paper reuses the representation the front-end already requires. We register scans against an Unsigned Distance Field (UDF) to replace the correspondence search of Iterative Closest Point (ICP) [12], [13] with constant-time lookups of the distance and its gradient [14]. Then, the local point cloud generating that field and the accumulated scans determine which returns are dynamic and what rasterizes into the final occupancy grid.

The main contribution of this paper is RicoSLAM, a 2D LiDAR graph SLAM framework in which a single UDF representation per keyframe supports tracking, dynamics removal, and mapping (Fig. 1). We achieve this with three mechanisms: an inlier ratio-based splitting criterion that commits a keyframe

once the tracker can no longer register incoming scans; a relocalization policy that reattaches the tracker on revisits to an existing node instead of extending the graph; and a ray tracing-based removal of dynamic returns, applied online on the local map and offline before rasterization. This approach allows us to bound the back-end by the explored area rather than by the duration of the run, to track without wheel odometry against a reference consistent with the latest observation, and to deliver a cleaner occupancy grid representation.

In sum, we make four key claims. Our approach is able to (i) estimate trajectories within 0.05 m Absolute Trajectory Error (ATE) on IILABS 3D [15] dataset without wheel odometry, which no baseline configuration achieves; (ii) reach that accuracy with one to two orders of magnitude fewer nodes and edges than the strongest baseline, and bound the graph by the explored area rather than trajectory length when relocalization is enabled; (iii) process the IILABS 3D sequences faster than baselines, once RicoSLAM’s visualizer is disabled; and (iv) deliver a cleaner occupancy grid representation. These claims are backed up by the paper and our experimental evaluation. Our implementation is available in a public repository¹.

II. RELATED WORK

A. 2D LiDAR SLAM systems

SLAM spans filtering, scan matching, and pose graph formulations. GMapping [16] estimates a grid map using a Rao-Blackwellized particle filter. However, each particle has its own map, and the number of particles required to close loops grows with environment complexity. In contrast, HectorSLAM [17] performs scan matching by aligning beam endpoints to an occupancy grid map, which keeps the method lightweight but leads to incremental drift.

Pose graph formulations separate a front-end that produces relative-pose constraints from a back-end that refines the entire trajectory as a factor graph, solved with optimizers such as g^2o [18] or `srrg2_solver` [19]. Cartographer [20] tracks each scan against a local map and closes loops through a branch-and-bound search. SLAM Toolbox [21] couples a scan matching front-end with a sparse pose graph back-end. These frameworks assume a static world, where inconsistent returns from moving objects may be integrated into the map, degrading the environment representation used for localization.

B. Distance field-based registration

Registering scans against implicit surface representations avoids the explicit correspondence search on ICP [12], [13] algorithms. Normal Distribution Transform (NDT) [22] models scans as a mixture of local Gaussians. Signed distance functions [23] support both scan registration and localization in 2D [24], [25], and large-scale 2D SLAM with truncated signed distance maps [26]. RicoSLAM adopts the unsigned variant: the front-end registers each scan against the UDF of the current keyframe with point-to-point and point-to-plane metrics, whose formulation and evaluation are presented

in [14]. This paper builds on that front-end and focuses on keyframe lifecycle, handling environment dynamics, loop closure, and pose graph maintenance.

C. Dynamics handling in 2D LiDAR SLAM

Handling moving objects is central to deploying SLAM in changing environments [2]. One way is to use observation consistency. Biswas and Veloso [27] separate long- and short-term features by matching distances. Boniardi *et al.* [28] detect reconfigurations against CAD models. Signed distance fields have been used to reject range outliers and absorb semi-static changes [24], [29]. Multi-session variants classify points by comparing aligned passes [5], [6]. Moreover, NDT variants augmented with occupancy modeling use the beam sensor model to distinguish transient from static structure [30]–[32]. Layered and temporal representations maintain maps with different dynamics levels [3], [4]. Semantic and object-detection approaches typically target visual or 3D LiDAR data and depend on trained classifiers [7], [8].

Other methods exploit ray tracing. Lázaro *et al.* [10] update map points using a 1-1 ray-casting model. Stefanini *et al.* [33] generalize [10] to 1-to-N. Pomerleau *et al.* [34] accumulate a Bayesian dynamic probability per point. Overall, RicoSLAM adopts the ray tracing approach of Lázaro *et al.* [10] while omitting their averaging step. In contrast to the literature, no additional map layer, prior map, or trained classifier is required. Online removal is confined to the active local map, bounding its cost and keeping the reference consistent with the observation, while sustaining localization in highly dynamic environments. The same rule is re-evaluated globally for offline dynamics removal, so that only surviving endpoints rasterize the final 2D occupancy grid delivered to navigation.

D. Bounded pose graphs and lifelong SLAM

Pose graph SLAM grows with trajectory length and operation time [2]. The most common strategy is to reduce a graph that has already grown. Kretschmar and Stachniss [9] retain the laser measurements with the highest expected information gain and marginalize the corresponding nodes, sparsifying the dense links with a Chow-Liu tree approximation. DPG-SLAM [4] removes nodes whose observations have become inconsistent with the current state of a low-dynamic environment. Lázaro *et al.* [10] condense measurements from revisited local maps to keep the graph compact across repeated traversals. RTAB-Map [11] bounds the working memory by transferring locations to a long-term store, excluding them from loop-closure search and optimization. Instead, RicoSLAM avoids redundant nodes altogether. The inlier ratio-based splitting criterion defers keyframe creation while the tracker holds the current keyframe, and the relocalization policy reattaches the tracker to the existing graph on revisits. Thus, graph size scales with the spatial extent of the explored environment rather than operation time, with no dedicated back-end sparsification.

III. RICOSLAM FRAMEWORK

RicoSLAM (Fig. 1) comprises a tracker front-end that registers each scan against the current keyframe [14], a factor-

¹<https://github.com/INESCTEC/ricoslam>

graph back-end [19] with a loop finder and a relocalization policy, and an offline occupancy-grid mapper that consumes the optimized graph. Each variable $\mathbf{x} \in SE(2)$ of the factor graph holds a keyframe's world-frame pose, local point cloud, precomputed UDF, and aggregated raw scans. Each factor encodes a geodesic relative-pose measurement produced either by odometric chaining of the tracker or by loop closure. The current world-frame pose is $\mathbf{x}_{R,W} = \mathbf{x}_{kf} \cdot \mathbf{x}_{R,kf}$, with $\mathbf{x}_{kf}$ the current keyframe estimate and $\mathbf{x}_{R,kf}$ the tracker's local robot pose, and all error and Jacobian derivations adopt the right-sided perturbation $\mathbf{x} \oplus \Delta\mathbf{x} = \mathbf{x} \cdot \exp(\Delta\mathbf{x})$.

A. Unsigned Distance Field (UDF) front-end

The tracker inherits the UDF-based front-end of [14]. Each scan becomes a 2D point cloud in the sensor frame, voxelized at the map resolution and aligned to the current keyframe's local map by minimizing the precomputed point-to-point residual over the $SE(2)$ manifold with a Gauss-Newton optimizer, replacing the per-iteration nearest-neighbor search of ICP [12], [13] with $O(1)$ grid lookups of the distance and its gradient. See [14] for the residual formulation, its Jacobian, and the distance map data structures. Two outputs feed the RicoSLAM back-end. At convergence, the aligner exposes a laser-odometry estimate $\mathbf{x}_{R,kf}$ and an *inlier ratio* $\eta_{inliers}$, later used as a triggering criterion. Wheel odometry only optionally initializes the aligner.

B. Keyframe maintenance

For every successfully registered scan, the tracker evaluates two criteria. A *merging* criterion integrates the latest cloud into the local map through the ray tracing-based merger of Sec. III-C. A *splitting* criterion commits the current keyframe to the pose graph and starts a new one. Both share a common interface and, for $SE(2)$ -displacement types, evaluate the accumulated motion via (1), where the reference pose $\mathbf{x}_{R,ref}$ is reset once per keyframe when splitting or after every merge.

$$\Delta = (\Delta_t^T, \Delta_\theta)^T = \log(\mathbf{x}_{R,ref}^{-1} \cdot \mathbf{x}_{R,kf}) \in \mathbb{R}^3 \quad (1)$$

Beyond classical fixed-displacement rules (*translation, rotation, viewpoint*), RicoSLAM introduces an *inlier ratio* criterion that triggers when $\eta_{inliers} < \eta_{max}$. It commits a keyframe only when the aligner can no longer register scans against the current local map. As shown in Sec. IV, this policy yields more compact graphs at no accuracy cost.

All criteria including *empty* expose a force-trigger interface through ROS services. Assigning split and merging criteria selects the operating mode. Turning off merging yields a *scan-based* graph SLAM mode, where the local map is never updated and each keyframe holds a single scan. Enabling it yields a *local map-based* mode, where several scans are aggregated before a split. Disabling splitting suppresses keyframe creation entirely, degenerating into a scan-to-map laser odometry. Assigning the *empty* type to both roles places keyframing under external control, allowing task-level policies to command split and merge events, as deployed for container-unloading

operations in CARGO [35]. The evaluation in Sec. IV uses the scan- and local map-based modes.

C. Ray tracing-based dynamics removal

Dynamic obstacles conflict with the static-world assumption of classical 2D SLAM. RicoSLAM addresses them with a ray-tracing mechanism at two stages: online, while maintaining the local map of the current keyframe, and offline, when rasterizing the final occupancy grid.

1) *Online removal in the local map*: On every merge, each map point $\mathbf{m}_i$ is expressed in the sensor frame of the current pose, $\mathbf{m}_i^{S_{curr}} = \mathbf{T}_R^S(\mathbf{x}_{R,kf})^{-1} \mathbf{T}_S^R \mathbf{m}_i$ with $\mathbf{T}_S^R$ the sensor pose w.r.t. the robot, and assigned to the ray index it would occupy in the latest scan data. Consistent with the single-return model, only the closest map point per ray is retained, with range $\rho_{M,k}$. For the current return $\mathbf{z}_k$ on ray index k , of range $\rho_{S,k}$ and range difference $\Delta\rho_k = \rho_{S,k} - \rho_{M,k}$, the per-ray update follows (2), whose thresholds $\tau_{new}, \tau_{through} > 0$ ignore noise near $\Delta\rho_k \approx 0$ and whose *see-through* case performs the online dynamics removal, replacing an outdated map point with the latest observation. After any update, the keyframe's UDF is rebuilt from the updated local cloud.

$$\begin{cases} \text{add } \tilde{\mathbf{z}}_k = \mathbf{T}_R^S \mathbf{x}_{R,kf} \mathbf{T}_S^R \mathbf{z}_k & \text{no map point on ray } k, \\ \text{add } \tilde{\mathbf{z}}_k & \text{if } \Delta\rho_k < -\tau_{new}, \\ \text{replace map point with } \tilde{\mathbf{z}}_k & \text{if } \Delta\rho_k > +\tau_{through}, \\ \text{keep map unchanged} & \text{otherwise} \end{cases} \quad (2)$$

This rule follows the 1-1 ray-casting model of Lázaro *et al.* [10], but omits its averaging step, following the principle of Biber and Duckett [3] that a map should contain only observed values, avoiding another point interpolation.

2) *Offline removal in occupancy grid*: RicoSLAM produces a 2D occupancy grid from the raw scans stored in the keyframes and the optimized graph, placing each scan $\mathcal{S}$ in the world frame with $\mathbf{x}_{S,W}^S = \mathbf{x}_{kf}^* \cdot \mathbf{x}_{R,kf}^S \cdot \mathbf{T}_S^R$. Before rasterization, an optional filtering stage reuses the merger of (2), now evaluated globally over all scans. So, discarded (dynamic) returns do not contribute to the occupancy statistics. Each grid cell accumulates hit/pass counters (n_{hit}, n_{pass}): every valid beam is traversed from sensor to endpoint with integer-only Bresenham rasterization, incrementing n_{pass} along the ray and n_{hit} at the endpoint cell, while beams exceeding the usable range are clipped and update only pass counters. Then, we compute occupancy probability via (3) and classify the cells into free/occupied/unknown. Offline filtering of dynamic returns sharpens the resulting distribution, concentrating cells near $p_{occ} \in \{0, 1\}$ instead of ambiguous mid-range values produced by inconsistent hit/pass evidence.

$$p_{occ}(r, c) = \frac{n_{hit}(r, c)}{n_{pass}(r, c)}, \quad n_{pass}(r, c) > 0 \quad (3)$$

D. Loop closure and relocalization

1) *Loop closure detection*: The loop finder in RicoSLAM combines geometric and topological proximity. Every variable $\mathbf{x}_c$ closer than $d_{\ell, max}$ to the current pose enters a short-loop

hint set $\mathcal{H}_s$, with the relative pose $\hat{\mathbf{x}}_{R,c} = \mathbf{x}_c^{-1} \cdot \mathbf{x}_{kf} \cdot \mathbf{x}_{R,kf}$ that propagates the robot into the candidate frame. Then, a graph visitor promotes a hint to the loop set $\mathcal{H}$ only when its topological distance exceeds $L_{\min}$, discarding candidates whose short chain of odometry edges would make the new constraint redundant. The loop aligner validates each hint in $\mathcal{H}$ by registering the latest cloud against the candidate's local map from two initial guesses: $\hat{\mathbf{x}}_{R,c}$ and $\hat{\mathbf{x}}_{\text{same}} = \exp((0, 0, \theta_{\hat{\mathbf{x}}_{R,c}})^T)$ that keeps the heading but places the robot at the candidate's origin. A successful registration returning $\hat{\mathbf{x}}_{R,c}^*$ adds an $SE(2)$ pose-pose factor with measurement $\mathbf{z} = \hat{\mathbf{x}}_{R,c}^* \cdot \mathbf{x}_{R,kf}^{-1}$ between the two keyframes. The solver [19] re-optimizes the graph incrementally whenever adding at least one loop factor.

2) *Relocalization*: Online graph SLAM grows without bound when the robot revisits a mapped area and incoming scans keep spawning keyframes that duplicate existing coverage. Instead of pruning such nodes afterwards by marginalization-based sparsification [2], RicoSLAM avoids creating them. When relocalization is enabled and $\mathcal{H}_s$ is non-empty, the loop aligner also registers the latest cloud into the local maps of the short-loop hints. The closest successfully registered candidate h_s^* is evaluated through the robot-to-candidate distance $d_{\text{loop}} = \|\mathbf{x}_{h_s^*}^{-1} \cdot \mathbf{x}_{R,W}\|_2$ and the robot-to-keyframe distance $d_{\text{local}} = \|\mathbf{t}_{R,kf}\|_2$. Whenever $d_{\text{loop}} < d_{\text{local}}$, RicoSLAM switches the current keyframe to h_s^* by resetting its local map to that candidate. Thus, subsequent scans register against an existing node instead of creating a new one. In scan mode, this tracks the robot within the existing graph. In local mode, it additionally merges new observations into an existing keyframe. Either way, the back-end graph is bounded by the spatial extent of the explored environment rather than by run duration or path length, even under repeated revisits.

E. Pose graph and pose history

1) *SE(2) pose-pose factor*: Odometric and loop-closure edges share the same $SE(2)$ geodesic factor. For keyframes $\mathbf{x}_{\text{from}}, \mathbf{x}_{\text{to}}$ connected by a measured relative pose $\mathbf{z}$, (4) gives the prediction $\hat{\mathbf{z}}$ and the residual $\mathbf{e}$, with Jacobians following the right-sided perturbation and provided by `srrg2_solver` [19]. The information matrix $\mathbf{\Omega}$ is either inherited from the aligner's approximate Hessian at convergence or set to a fixed diagonal, typically more confident in heading than in translation.

$$\hat{\mathbf{z}} = \mathbf{x}_{\text{from}}^{-1} \cdot \mathbf{x}_{\text{to}} \in SE(2), \quad \mathbf{e} = \log(\mathbf{z}^{-1} \cdot \hat{\mathbf{z}}) \in \mathbb{R}^3 \quad (4)$$

2) *Online and offline pose streams*: Finally, RicoSLAM logs two pose streams. The *online* stream reports the world-frame pose at scan time, $\mathbf{x}_{R,W}^{\text{online}}(t) = \mathbf{x}_{kf}(t) \cdot \mathbf{x}_{R,kf}(t)$, from the back-end estimate $\mathbf{x}_{kf}(t)$. The *offline corrected* stream stores, per processed scan, only the keyframe identifier $\text{id}_{kf}(t)$ and the online local pose $\mathbf{x}_{R,kf}(t)$. Then, RicoSLAM reconstructs world-frame poses *a posteriori* from the optimized keyframe estimates once the run completes, as in (5). Comparing both streams isolates the contribution of the back-end optimization and loop closures to the final trajectory.

$$\mathbf{x}_{R,W}^{\text{offline}}(t) = \mathbf{x}_{\text{id}_{kf}(t)}^*(t) \cdot \mathbf{x}_{R,kf}(t) \quad (5)$$

IV. EXPERIMENTAL RESULTS

Our experiments are designed to support the four claims of Sec. I on two evaluations. First, the IILABS 3D [15] dataset assesses accuracy without wheel odometry (Sec. IV-B), bounded graph growth (Sec. IV-C), and computational cost (Sec. IV-D). However, IILABS 3D covers a mostly static environment. Consequently, the second evaluation uses two sequences recorded in industrial sites, Flowbotic and Kuka, to support the claim on dynamics removal (Sec. IV-E). These provide neither a ground-truth trajectory nor a static reference map. So, Sec. IV-E focuses on the final map representation.

A. IILABS 3D experimental setup

RicoSLAM is evaluated on four planar sequences of IILABS 3D [15], which provides wheel odometry, Hokuyo UST-10LX 2D laser data, and ground truth. *Nav A Diff* (275 m, 755 s) and *Nav A Omni* (112 m, 387 s) cover the same space under differential-drive and holonomic motion, respectively. *Loop* (232 m, 620 s) leaves and re-enters that space through different doors via an external corridor, triggering loop closure. Lastly, *Slippage* (39 m, 93 s) follows a straight path with high-speed lateral motion inducing wheel slippage.

We evaluate five odometry-free configurations of RicoSLAM, differing only in the keyframe split and merge criteria and in whether relocalization or factor information-matrix updates are enabled, where rows marked † report the offline corrected stream of (5), not a separate online system:

- **scan**: a 50 % inlier-ratio splitting criterion with merging disabled, so every scan registered into the back-end graph becomes its own keyframe;
- **scan w/reloc.**: as *scan*, with relocalization. Since it reuses the loop aligner to decide whether to switch the current keyframe, the inlier-ratio failure threshold is raised from 30 % to 70 % to avoid swapping keyframes without sufficient observation overlap;
- **local**: local map mode without relocalization, splitting on a 2.50 m translation threshold for uniform keyframe spacing and merging above an 80 % inlier ratio, with $\tau_{\text{new}} = \tau_{\text{through}} = 0.20$ m (see Sec. III-C);
- **local w/upd.**: as *local*, additionally re-evaluating information matrices of the incident factors by aligning their local maps at each split, given that the current keyframe accumulates information until its successor spawns;
- **local w/reloc.**: as *local*, with relocalization and the minimum graph distance to relocalize reduced from 5 to 3 edges, since fewer keyframes are produced.

We compare RicoSLAM against GMapping [16], SLAM Toolbox [21], and Cartographer [20]. We vary the update thresholds of the first two within a convergent range: 0.3 m 0.3 rad up to 1.0 m 1.0 rad for [16], and 0.3 m to 0.5 m for [21]. Cartographer runs under its default thresholds, 0.5 m 0.5 rad, and 1.0 m 1.0 rad, with the number of scans per submap and the optimization period set to 90, 20, and 10, respectively. Its *scan-only* variants are laser-driven, *scan+wheel* adds wheel odometry as a motion prior, and * replaces the default fast matcher with the real-time correlative one.

Cartographer’s grid is kept at the default 0.05 m resolution, as the 0.03 m used for all other systems did not converge.

Trajectories are converted to TUM format and evaluated with `evo` [36]: ATE as Root Mean Square Error (RMSE) with its standard deviation σ_{ATE} , and relative translational (RTE) and rotational (RRE) errors over 10 m segments, matching the dataset’s own protocol [15]. All systems process the bags on the same laptop (AMD Ryzen 9 7945HX, 16 cores / 32 threads, 32 GiB of RAM), each built from source in a single Ubuntu 20.04 / ROS Noetic container via Docker.

B. Trajectory accuracy

Table I summarizes all configurations. GMapping is the least accurate method on three sequences, with a best-configuration ATE of 0.117 m, 0.118 m, and 0.113 m on *Nav A Diff*, *Nav A Omni*, and *Loop*, respectively. This result follows from propagating wheel odometry between scan updates and from reading the estimate off the best particle of the filter, making the estimated pose to jump. *Slippage* is the exception, where GMapping has lower relative error than SLAM Toolbox (1.85 % and $0.272^\circ \text{m}^{-1}$ against 5.13 % and $0.326^\circ \text{m}^{-1}$).

SLAM Toolbox improves on GMapping elsewhere, lowering the ATE up to 0.053 m, 0.061 m, and 0.040 m on *Nav A Diff*, *Nav A Omni*, and *Loop*, respectively. Still, it integrated wheel odometry between scan matches into its back-end, in contrast to RicoSLAM, which updates the robot pose by registering every incoming scan. The lateral manoeuvres of *Slippage* are the ones the dataset authors measured as most damaging to the platform’s odometry [15]. There, the method degrades up to 6.00 % RTE and $0.487^\circ \text{m}^{-1}$ RRE, while scan-driven methods stay below 0.83 % and $0.139^\circ \text{m}^{-1}$.

Cartographer is the strongest baseline in Table I. The best ATE on *Nav A Diff* (0.046 m), *Nav A Omni* (0.023 m), and *Loop* (0.025 m) are obtained with wheel odometry. Moreover, all five $\{-\}$ entries are runs with its fast matcher. The results are separate by which prior is available. Without odometry, Cartographer loses track of *Loop* once the thresholds are coarsened. With odometry, it diverges on *Slippage* at every threshold, as the prior it relies on is the one the slip corrupts. Thus, Cartographer relies on the real-time correlative matcher, which is also its most expensive component (Sec. IV-D).

Online and without odometry, RicoSLAM is on par for ATE with the best Cartographer configuration on *Nav A Diff* (0.043 m against 0.046 m), *Loop* (0.029 m against 0.025 m), and *Slippage* (0.013 m). Restricted to odometry-free operation, the best RicoSLAM variant matches or improves on the best Cartographer *scan-only* setting on all four sequences. *Nav A Omni* is the only sequence where the odometry-aided baseline keeps an edge (0.023 m against 0.030 m), yet no baseline configuration stays below 0.05 m on all four sequences, whereas both *scan* and *local* modes of RicoSLAM do.

The offline corrected stream (RicoSLAM[†]) is the most accurate overall by expressing the trajectory against the optimized graph once loop closures are applied via (5). In *scan* mode, the ATE halves on *Nav A Diff* (0.043 m to 0.022 m) and *Nav A Omni* (0.049 m to 0.023 m) and drops by a third on

Loop (0.029 m to 0.019 m), with no gain on the 39 m *Slippage* run. RTE is unchanged, since the 10 m relative metrics are dominated by the front-end tracker shared by both streams, while rotational drift improves consistently. The residual error is also nearly uniform along the trajectory. σ_{ATE} stays below 0.020 m for every configuration and sequence, indicating that the RicoSLAM[†] estimate is limited by systematic sources (extrinsic sensor calibration, ground-truth registration, and trajectory alignment) rather than by framework itself.

Relocalization trades accuracy for graph size. Relocalizing against a local map built earlier in the run degrades the online ATE results, from 0.029 m to 0.079 m on *Loop* in *scan* mode and from 0.045 m to 0.093 m on *Nav A Diff* in *local* mode, although every variant stays below 0.10 m. The back-end absorbs most of this cost. In the RicoSLAM[†] stream, *local w/reloc.* matches *local* within 0.001 m on *Nav A Diff*, *Nav A Omni*, and *Slippage*.

C. Scalability analysis

Table II and Table III report graph size across the four sequences. For the pose-graph methods, we list the number of scans, graph nodes (variables), and edges. Although Cartographer builds submaps, every scan still enters the graph as a node. For GMapping, only the scan count is meaningful, as the particle filter carries no explicit graph. Overall, every RicoSLAM variant yields a far smaller graph than any baseline while retaining the accuracy of Sec. IV-B. For example, on *Nav A Diff*, the *scan* mode needs 77 nodes against Cartographer’s 543 at its most accurate setting.

The two RicoSLAM modes differ in their splitting criteria. The information-driven split on *scan* mode commits fewer variables than the translation split on *local* mode for every sequence, at comparable accuracy. Cartographer, the strongest accuracy baseline, reaches that accuracy with one to two orders of magnitude more nodes and edges than RicoSLAM.

As for the relocalization policy, a core contribution of this paper, it bounds the graph even further than the plain *scan* and *local* variants. Once the robot recognizes previously mapped spaces, it relocalizes within the existing graph instead of extending it, reducing the node count on revisited trajectories. On *Loop*, a five-cycle run, *scan* mode with relocalization drops from 83 to 19 nodes and *local* from 89 to 18. This indicates that graph size tracks the spatial extent of the environment rather than the length of the trajectory.

D. Execution time

Table IV reports the total processing time. Rather than replaying each recording at sensor rate, every framework processes the serialized messages as fast as possible. So, the reported time reflects computation alone. GMapping and RicoSLAM run their SLAM system on a single thread, while Cartographer spreads across several. SLAM Toolbox is excluded, as the algorithm exposes no equivalent offline mode.

Cartographer with its fast matcher records the lowest times, even though not reaching its best accuracy (Sec. IV-B). Once using the correlative matcher, all configurations are slower

TABLE I
TRAJECTORY ACCURACY EVALUATION OF SLAM FRAMEWORKS ON THE IILABS 3D [15] DATASET

Method	Variant	Nav A Diff				Nav A Omni				Loop				Slippage			
		ATE (m)	σ_{ATE} (m)	RTE (%)	RRE (°/m)	ATE (m)	σ_{ATE} (m)	RTE (%)	RRE (°/m)	ATE (m)	σ_{ATE} (m)	RTE (%)	RRE (°/m)	ATE (m)	σ_{ATE} (m)	RTE (%)	RRE (°/m)
GMapping [16]	0.3m 0.3rad	0.148	0.069	1.77	0.111	0.126	0.046	2.16	0.093	0.122	0.075	3.04	0.117	0.055	0.025	1.85	0.310
	0.5m 0.5rad	0.127	0.047	2.01	0.120	0.122	0.038	1.72	0.116	0.114	0.068	3.67	0.129	0.048	0.021	3.49	0.272
	1.0m 1.0rad	0.117	0.030	2.04	0.167	0.118	0.029	2.08	0.170	0.113	0.069	3.25	0.148	0.061	0.031	4.19	0.320
SLAM Toolbox [21]	0.3m	0.075	0.046	0.86	0.084	0.081	0.044	1.40	0.105	0.068	0.037	1.69	0.061	0.042	0.021	5.13	0.326
	0.5m	0.053	0.027	0.94	0.121	0.061	0.025	1.89	0.175	0.040	0.021	1.68	0.055	0.064	0.029	6.00	0.487
Cartographer [20] (default)	scan-only	0.113	0.071	1.01	0.078	0.033	0.017	0.72	0.071	0.119	0.083	1.42	0.068	0.015	0.005	0.80	0.111
	scan-only*	0.072	0.050	1.09	0.087	0.029	0.015	0.88	0.073	0.029	0.015	0.99	0.076	<u>0.013</u>	<u>0.005</u>	0.34	0.042
	scan+wheel	0.098	0.060	1.04	0.091	0.030	0.014	0.98	0.090	0.114	0.085	1.23	0.059	—	—	—	—
	scan+wheel*	0.069	0.046	1.17	0.104	0.033	0.015	0.77	0.070	0.025	0.013	1.07	0.083	0.055	0.031	0.43	0.097
Cartographer [20] (0.5m 0.5rad)	scan-only	0.047	0.025	0.99	0.079	0.034	0.020	0.79	0.076	—	—	—	—	0.017	0.007	0.70	0.069
	scan-only*	0.060	0.035	0.99	0.081	0.034	0.018	0.86	0.075	0.131	0.106	1.56	0.083	0.015	0.007	0.48	0.058
	scan+wheel	0.046	0.028	1.01	0.098	0.023	0.009	0.77	0.065	0.101	0.083	1.32	0.071	—	—	—	—
	scan+wheel*	0.046	0.026	0.84	0.074	0.031	0.017	0.82	0.068	0.111	0.086	1.49	0.074	0.044	0.026	0.49	0.074
Cartographer [20] (1.0m 1.0rad)	scan-only	0.092	0.052	1.10	0.083	0.038	0.019	0.87	0.067	—	—	—	—	0.020	0.010	0.79	0.139
	scan-only*	0.091	0.052	1.00	0.083	0.062	0.032	0.84	0.077	0.182	0.106	2.08	0.089	0.020	0.008	0.62	0.050
	scan+wheel	0.059	0.029	1.02	0.100	0.025	0.010	0.78	0.061	0.091	0.053	1.14	0.057	—	—	—	—
	scan+wheel*	0.061	0.034	1.03	0.083	0.032	0.015	0.93	0.078	0.154	0.081	1.88	0.078	0.050	0.031	0.39	0.061
RicoSLAM	scan	0.043	0.026	0.82	0.074	0.049	0.028	0.92	0.078	0.029	0.017	0.95	0.056	0.014	0.006	0.66	0.057
	scan w/reloc.	0.062	0.037	1.02	0.088	0.074	0.049	1.07	0.099	0.079	0.071	1.66	0.081	0.013	0.005	0.68	0.059
	local	0.045	0.019	0.91	0.077	0.042	0.018	0.98	0.082	0.039	0.015	0.94	0.062	0.015	0.006	0.83	0.083
	local w/upd.	0.056	0.037	0.88	0.074	0.030	0.015	0.95	0.076	0.032	0.014	0.89	0.055	0.015	0.006	0.63	0.062
RicoSLAM†	local w/reloc.	0.093	0.046	0.87	0.070	0.077	0.028	0.92	0.080	0.042	0.018	1.24	0.048	0.015	0.007	0.75	0.062
	scan	0.022	0.009	0.85	0.065	0.023	0.010	0.84	0.073	0.019	0.008	1.13	0.046	0.015	0.005	0.70	0.050
	scan w/reloc.	0.043	0.017	1.08	0.082	0.045	0.020	1.04	0.086	0.021	0.010	1.09	0.047	0.013	0.005	0.68	0.059
	local	0.034	0.015	0.84	0.067	0.032	0.016	0.94	0.071	0.029	0.010	1.06	0.046	0.016	0.006	0.69	0.064
	local w/upd.	0.027	0.012	0.86	0.063	0.026	0.012	0.95	0.068	0.026	0.009	1.09	0.046	0.015	0.006	0.70	0.080
RicoSLAM†	local w/reloc.	0.034	0.014	0.79	0.061	0.031	0.016	0.89	0.077	0.038	0.015	1.28	0.042	0.015	0.007	0.75	0.062

Bold indicates best of online vs offline within 0.005 m (ATE), 0.05 % (RTE), 0.005 ° m⁻¹ (RRE). Underlined indicates best within variants.

* enable the real-time correlative scan matcher of Cartographer [20], instead of the default fast version for the matching process.

† offline corrected pose stream of the configurations from the online RicoSLAM results.

TABLE II

SCALABILITY RESULTS OF SLAM FRAMEWORKS ON THE IILABS 3D [15] DATASET (NAV A DIFF, NAV A OMNI)

Method	Variant	Nav A Diff			Nav A Omni		
		#node	#scan	#edge	#node	#scan	#edge
GMapping [16]	0.3m 0.3rad	—	897	—	—	373	—
	0.5m 0.5rad	—	544	—	—	226	—
	1.0m 1.0rad	—	273	—	—	113	—
SLAM Toolbox [21]	0.3m	541	541	2488	222	222	409
	0.5m	310	310	470	126	126	163
Cartographer [20] (default)	scan-only	2814	2814	14 812	1629	1629	5836
	scan-only*	2810	2810	14 736	1616	1616	5645
	scan+wheel	2817	2817	14 766	1636	1636	5832
	scan+wheel*	2810	2810	14 669	1625	1625	5844
Cartographer [20] (0.5m 0.5rad)	scan-only	544	544	2622	222	222	664
	scan-only*	544	544	2603	222	222	661
	scan+wheel	543	543	2652	222	222	659
	scan+wheel*	544	544	2627	222	222	656
Cartographer [20] (1.0m 1.0rad)	scan-only	273	273	1128	112	112	290
	scan-only*	273	273	1146	112	112	288
	scan+wheel	273	273	1153	112	112	290
	scan+wheel*	273	273	1152	112	112	286
RicoSLAM	scan	77	77	173	30	30	46
	scan w/reloc.	22	22	22	18	18	18
	local	106	376	248	43	158	66
	local w/upd.	106	376	246	43	158	66
	local w/reloc.	20	96	29	16	79	19

TABLE III

SCALABILITY RESULTS OF SLAM FRAMEWORKS ON THE IILABS 3D [15] DATASET (LOOP, SLIPPAGE)

Method	Variant	Loop			Slippage		
		#node	#scan	#edge	#node	#scan	#edge
GMapping [16]	0.3m 0.3rad	—	757	—	—	141	—
	0.5m 0.5rad	—	459	—	—	84	—
	1.0m 1.0rad	—	231	—	—	42	—
SLAM Toolbox [21]	0.3m	458	458	1291	77	77	111
	0.5m	261	261	393	46	46	59
Cartographer [20] (default)	scan-only	2455	2455	11 213	204	204	326
	scan-only*	2437	2437	10 777	197	197	310
	scan+wheel	2455	2455	11 202	—	—	—
	scan+wheel*	2452	2452	11 111	196	196	307
Cartographer [20] (0.5m 0.5rad)	scan-only	—	—	—	77	77	145
	scan-only*	460	460	2073	77	77	145
	scan+wheel	460	460	2085	—	—	—
	scan+wheel*	460	460	2074	77	77	145
Cartographer [20] (1.0m 1.0rad)	scan-only	—	—	—	39	39	73
	scan-only*	231	231	908	39	39	74
	scan+wheel	231	231	953	—	—	—
	scan+wheel*	231	231	927	39	39	73
RicoSLAM	scan	83	83	155	10	10	11
	scan w/reloc.	19	19	19	8	8	7
	local	89	328	168	15	48	17
	local w/upd.	89	328	168	15	48	17
	local w/reloc.	18	83	18	8	34	7

than every RicoSLAM run that keeps the OpenGL global-map visualizer active. GMapping is the most expensive baseline. Its time scales inversely with coarsening the linear and angular update thresholds, which set how often a scan is integrated and every particle updated. Disabling the visualizer in RicoSLAM

makes it the fastest framework on all four sequences, undercutting even Cartographer's fast matcher while retaining the accuracy of Sec. IV-B. Within RicoSLAM, the relocalization variants (*w/reloc.*) run longer than their plain counterparts, since the loop aligner performs additional registrations to

TABLE IV
TOTAL EXECUTION TIME EVALUATION OF
SLAM FRAMEWORKS ON THE IILABS 3D [15] DATASET

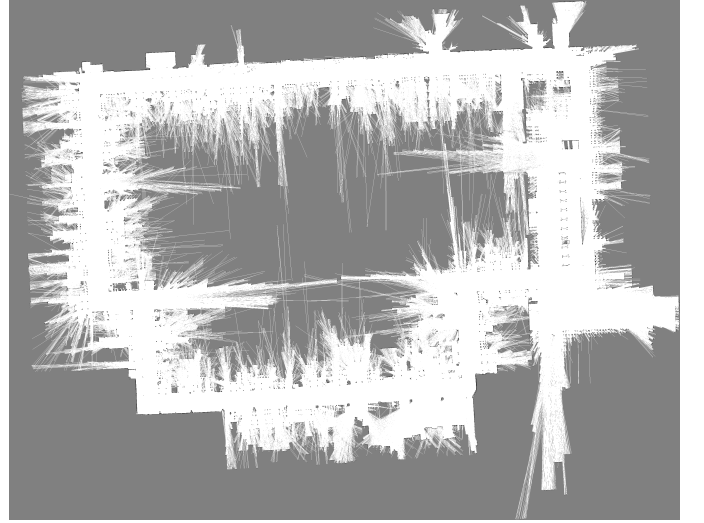
Method	Variant	Nav A Diff (s)	Nav A Omni (s)	Loop (s)	Slippage (s)
GMapping [16]	0.3m 0.3rad	673.742	278.550	582.317	102.783
	0.5m 0.5rad	407.757	168.363	352.890	62.899
	1.0m 1.0rad	207.233	85.411	177.995	32.533
Cartographer [20] (default)	scan-only	46.666	18.642	43.745	3.792
	scan-only*	139.121	70.536	121.174	19.091
	scan+wheel	49.105	20.261	45.486	—
	scan+wheel*	142.860	74.093	124.660	19.635
Cartographer [20] (0.5m 0.5rad)	scan-only	28.632	13.964	—	3.272
	scan-only*	129.761	67.024	112.056	18.914
	scan+wheel	32.776	16.205	29.196	—
	scan+wheel*	134.169	69.024	116.307	19.597
Cartographer [20] (1.0m 1.0rad)	scan-only	26.859	13.391	—	3.257
	scan-only*	127.548	65.976	109.503	18.612
	scan+wheel	31.217	15.743	26.563	—
	scan+wheel*	132.227	68.418	113.216	19.424
RicoSLAM	scan	82.978	41.475	67.406	10.401
	scan w/reloc.	101.336	48.628	75.116	10.099
	local	90.174	41.374	70.339	9.556
	local w/upd.	91.164	41.636	70.915	9.643
	local w/reloc.	113.148	50.272	77.608	10.841
RicoSLAM (no OpenGL)	scan	20.378	11.344	16.967	2.631
	scan w/reloc.	48.259	21.872	32.297	3.765
	local	25.320	12.389	20.008	3.084
	local w/upd.	26.110	12.464	20.581	3.125
	local w/reloc.	58.099	22.934	33.771	4.565

relocalize onto an existing node. This overhead is the cost of the bounded graphs reported in Sec. IV-C.

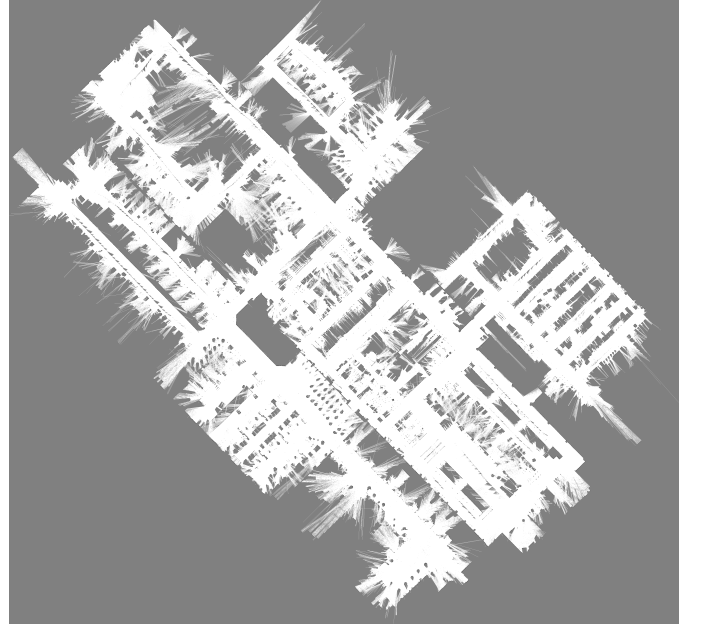
E. Dynamic environments

Lastly, we evaluate the offline dynamics removal of Sec. III-C on two industrial sequences: *Flowbotic 20251024*, a 289.5m run of 15m43s recorded by a mobile robot where moving trolleys and manually operated machinery coexist with the static infrastructure, and *Kuka*, a 2610.2m run of 01h18m19s with several long loops. RicoSLAM operates in local map mode using the online filtering to merge new observations onto the local representation, summarizing 1044 and 3988 scans into 103 and 490 keyframes, respectively. Both grids are rasterized at 0.03 m with $\tau_{\text{new}} = \tau_{\text{through}} = 0.10$ m and compared against a *standard* variant that rasterizes the same scans with the offline filtering stage disabled. Fig. 2 presents the 2D occupancy grids produced by RicoSLAM with offline filtering for illustration purposes.

The offline merger discards 21.2 % (134 475 of 633 661) and 23.3 % (417 954 of 1 794 718) of range-valid returns on the *Flowbotic 20251024* and *Kuka* sequences, respectively, which reaches the occupancy grid in two ways. First, 46 445 and 115 602 cells revert to unknown, as their only supporting beams were dynamic returns. Second, the distribution over the cells that remain observed sharpens. Writing the middle mass $m_{\text{mid}}(\gamma) = \Pr[p_{\text{occ}} \in (\gamma, 1 - \gamma)]$ for the fraction of observed cells with weak free/occupied evidence, filtering lowers it from 3.17 % to 2.47 % and from 3.73 % to 2.94 % at $\gamma = 0.05$ on the *Flowbotic 20251024* and *Kuka* sequences, moving cells to a more decided state. The same ordering holds at $\gamma = 0.01$, from 4.47 % to 2.97 % and from 4.30 % to 3.11 %.



(a) Flowbotic 20251024



(b) Kuka

Fig. 2: 2D occupancy grid map produced by RicoSLAM in local map mode with the offline dynamics removal of Sec. III-C at 0.03 m resolution.

Still, the dynamics removal mechanism of Sec. III-C only discards a return when a later beam traverses the same ray and sees through it. Thus, objects in areas that the robot never revisits from another viewpoint may persist on the point cloud representation of the environment, as do the ground returns caused by imperfect sensor levelling on the *Flowbotic* dataset. Overcoming this limitation is left as future work. A playlist with experiments from Sec. IV is available online².

²https://www.youtube.com/playlist?list=PLvp8fJUEPxyR6GQ2JIyaQezfa_YsO3M4T

V. CONCLUSIONS

We presented RicoSLAM, a 2D LiDAR graph SLAM framework for dynamic environments built on a single UDF per keyframe, which serves as the tracker's registration target, as the source of the ray tracing that discards dynamic returns, and as the reference for reattaching to existing nodes on revisits. Scans can register without the wheel odometry prior, and a new variable enters the graph once the registration degrades, which bounds the back-end by the spatial extent of the environment rather than by the duration of the run. We evaluated the framework against GMapping [16], SLAM Toolbox [21], and Cartographer [20] on four ground-truthed sequences and two industrial datasets. RicoSLAM stayed below 0.05 m on all four sequences of the IILABS 3D [15] dataset, doing so with 77 graph nodes where the most accurate baseline needs 543. The offline dynamics removal filtering sharpens the occupancy distribution and reverts to unknown the cells supported only by dynamic returns.

Three limitations remain for future work. A return is discarded only once a later beam sees through it. Relocalization bounds the graph at a cost in online accuracy that a more selective switching rule should recover. Lastly, the dynamics removal mechanism targets transient dynamics within a single run, leaving semi-static changes of long-term deployments undistinguished from moving objects.

REFERENCES

- [1] "ISO 3691-4:2023: industrial trucks – safety requirements. part 1: driverless industrial trucks and their systems," standard, International Organization for Standardization (ISO), 2023.
- [2] R. Sousa, H. Sobreira, and A. Moreira, "A systematic literature review on long-term localization and mapping for mobile robots," *Journal of Field Robotics*, vol. 40, no. 5, pp. 1245–1322, 2023.
- [3] P. Biber and T. Duckett, "Experimental analysis of sample-based maps for long-term SLAM," *International Journal of Robotics Research*, vol. 28, no. 1, pp. 20–33, 2009.
- [4] A. Walcott-Bryant, M. Kaess, H. Johannsson, and J. Leonard, "Dynamic pose graph SLAM: long-term mapping in low dynamic environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pp. 1871–1878, 2012.
- [5] G. Kim and A. Kim, "LT-mapper: a modular framework for LiDAR-based lifelong mapping," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pp. 7995–8002, 2022.
- [6] L. Yang, S. Prakhya, S. Zhu, and Z. Liu, "Lifelong 3D mapping framework for hand-held & robot-mounted LiDAR mapping systems," *IEEE Robot. Autom. Lett.*, vol. 9, no. 11, pp. 9446–9453, 2024.
- [7] B. Bescos, J. F  cil, J. Civera, and J. Neira, "DynaSLAM: tracking, mapping, and inpainting in dynamic scenes," *IEEE Robot. Autom. Lett.*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [8] X. Ding, Y. Wang, R. Xiong, D. Li, L. Tang, H. Yin, and L. Zhao, "Persistent stereo visual localization on cross-modal invariant map," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 11, pp. 4646–4658, 2020.
- [9] H. Kretschmar and C. Stachniss, "Information-theoretic compression of pose graphs for laser-based SLAM," *International Journal of Robotics Research*, vol. 31, no. 11, pp. 1219–1230, 2012.
- [10] M. L  zaro, R. Capobianco, and G. Grisetti, "Efficient long-term mapping in dynamic environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pp. 153–160, 2018.
- [11] M. Labb   and F. Michaud, "RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.
- [12] P. Besl and N. McKay, "A method for registration of 3-D shapes," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 14, no. 2, pp. 239–256, 1992.
- [13] Y. Chen and G. Medioni, "Object modeling by registration of multiple range images," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, vol. 3, pp. 2724–2729, 1991.
- [14] R. Sousa, G. Grisetti, H. Sobreira, C. Silva, and A. Moreira, "Unsigned distance maps on 2D point cloud registration," in *Proc. Iberian Robot. Conf. (ROBOT)*, pp. 1–8, 2026.
- [15] J. Ribeiro, R. Sousa, J. Martins, A. Aguiar, F. dos Santos, and H. Sobreira, "Indoor benchmark of 3-D LiDAR SLAM at iilab – industry and innovation laboratory," *IEEE Access*, vol. 13, pp. 212421–212442, 2025.
- [16] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with Rao-Blackwellized particle filters," *IEEE Trans. Robot.*, vol. 23, no. 1, pp. 34–46, 2007.
- [17] S. Kohlbrecher, O. von Stryk, J. Meyer, and U. Klingauf, "A flexible and scalable SLAM system with full 3D motion estimation," in *Proc. IEEE Int. Symp. Saf., Secur., Rescue Robot. (SSRR)*, pp. 155–160, 2011.
- [18] R. K  mmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, "g2o: a general framework for graph optimization," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pp. 3607–3613, 2011.
- [19] G. Grisetti, T. Guadagnino, I. Aloise, M. Colosi, B. Della Corte, and D. Schlegel, "Least squares optimization: from theory to practice," *Robotics*, vol. 9, no. 3, p. 51, 2020.
- [20] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LIDAR SLAM," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pp. 1271–1278, 2016.
- [21] S. Macenski and I. Jambrecic, "SLAM toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.
- [22] P. Biber and W. Strasser, "The normal distributions transform: a new approach to laser scan matching," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 3, pp. 2743–2748, 2003.
- [23] B. Curless and M. Levoy, "A volumetric method for building complex models from range images," in *Proc. Annu. Conf. Comput. Graph. Interact. Techn. (SIGGRAPH)*, pp. 303–312, 1996.
- [24] M. Zhang, Y. Chen, and M. Li, "SDF-Loc: signed distance field based 2D relocalization and map update in dynamic environments," in *Proc. Amer. Control Conf. (ACC)*, pp. 1997–2004, 2019.
- [25] A. Millane, H. Oleynikova, C. Lanegger, J. Delmerico, J. Nieto, R. Siegwart, M. Pollefeys, and C. Cadena, "Freasures: localization in signed distance function maps," *IEEE Robot. Autom. Lett.*, pp. 1–1, 2021.
- [26] K. Daun, S. Kohlbrecher, J. Sturm, and O. von Stryk, "Large scale 2D laser SLAM using truncated signed distance functions," in *Proc. IEEE Int. Symp. Saf., Secur., Rescue Robot. (SSRR)*, pp. 222–228, 2019.
- [27] J. Biswas and M. Veloso, "Episodic non-Markov localization," *Robotics and Autonomous Systems*, vol. 87, pp. 162–176, 2017.
- [28] F. Boniardi, T. Caselitz, R. K  mmerle, and W. Burgard, "A pose graph-based localization system for long-term navigation in CAD floor plans," *Robotics and Autonomous Systems*, vol. 112, pp. 84–97, 2019.
- [29] J. Fu, C. Lin, Y. Taguchi, A. Cohen, Y. Zhang, S. Mylabathula, and J. Leonard, "PlaneSDF-based change detection for long-term dense mapping," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 9667–9674, 2022.
- [30] J. Saarinen, H. Andreasson, T. Stoyanov, and A. Lilienthal, "3D normal distributions transform occupancy maps: an efficient representation for mapping in dynamic environments," *International Journal of Robotics Research*, vol. 32, no. 14, pp. 1627–1644, 2013.
- [31] E. Einhorn and H.-M. Gross, "Generic 2D/3D SLAM with NDT maps for lifelong application," in *Proc. Eur. Conf. Mobile Robots (ECMR)*, pp. 240–247, 2013.
- [32] E. Einhorn and H.-M. Gross, "Generic NDT mapping in dynamic environments and its application for lifelong SLAM," *Robotics and Autonomous Systems*, vol. 69, pp. 28–39, 2015.
- [33] E. Stefanini, E. Ciancolini, A. Settimi, and L. Pallottino, "Safe and robust map updating for long-term operations in dynamic environments," *Sensors*, vol. 23, no. 13, p. 6066, 2023.
- [34] F. Pomerleau, P. Kr  si, F. Colas, P. Furgale, and R. Siegwart, "Long-term 3D map maintenance in dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pp. 3712–3719, 2014.
- [35] M. Lopes, A. Cordeiro, R. Sousa, J. Be  a, P. Costa, J. Souza, and M. Silva, "CARGO: container-unloading autonomous robot with generalized operation," in *Proc. Int. Conf. Autom., Robot. Appl. (ICARA)*, pp. 274–279, 2026.
- [36] M. Grupp, "evo: Python package for the evaluation of odometry and SLAM," 2017. URL: <https://github.com/michaelgrupp/evo>.